



CS340 : Algorithms and Data Structures

Sorting Examples

Dr. Ali Moltajaei Farid
Dept. of Computer Science

Example Codes

- Sorting codes and examples:

<https://www.softwaretestinghelp.com/sorting-techniques-in-cpp/>

How to Use Titan?

<https://urcourses.uregina.ca/mod/kalvidres/view.php?id=1676972>

Heap Sort

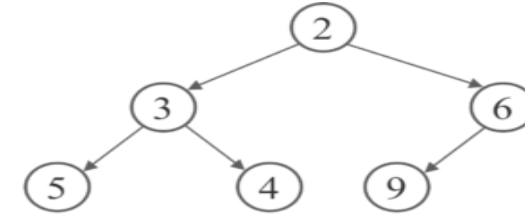
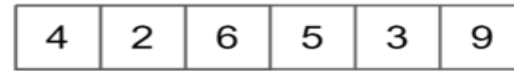
- Heapsort is a comparison-based sorting method based on the [binary heap](#) data structure. The binary heap structure allows the Heapsort algorithm to take advantage of the heap's properties:
 - Every node's value must be **greater (or smaller) than all values stored in its children**
 - It's a **complete tree**, which means it has the smallest possible height

Heap Sort

- We can summarise Heapsort into four main steps:
 - Build a min (or max) heap from the input array
 - At this point, the smallest item is stored at the root of the heap. We'll remove the element from the root node, and store the rightmost leaf in the root node.
 - Heapify the root of the tree
 - Repeat steps 2 and 3 while the size of the heap is greater than 1

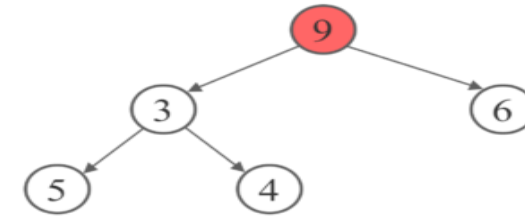
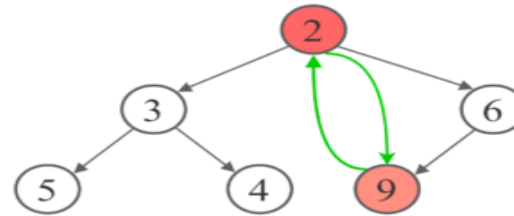
Heap Sort Example

Step 1: Build a min heap



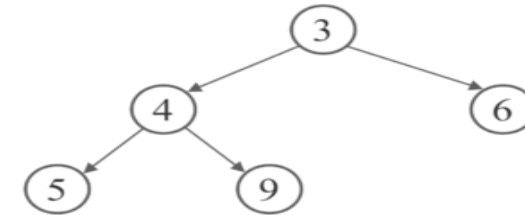
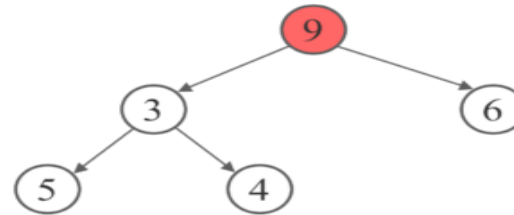
Sorted array = []

Step 2: Remove the root node



Sorted array = [2]

Step 3: Heapify



Sorted array = [2]

...

Step 4: Repeat step 2 and 3



Sorted array = [2, 3, 4, 5, 6, 9]

Quick Sort

- In average, as well as best cases, the time complexity of the Quicksort algorithm is $O(n \log n)$.
- In principle, the worst-case time complexity can be $O(n^2)$ if we select a bad pivot, and it may happen when the array is already sorted in ascending or descending order.

Quick Sort Example

44	75	23	43	55	12	64	77	33
----	----	----	----	----	----	----	----	----

pivot

12	33	23	43	44	55	64	77	75
----	----	----	----	----	----	----	----	----

12	33	23	43
----	----	----	----

55	64	77	75
----	----	----	----

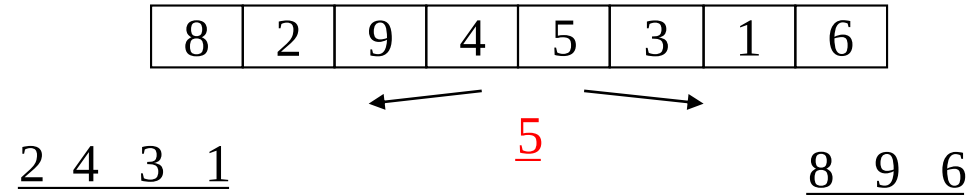
12	23	33	43
----	----	----	----

75	77
----	----

Quick Sort Pivots

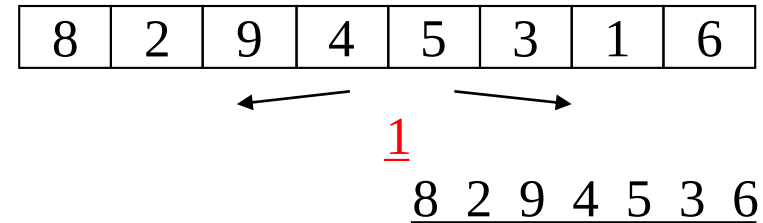
- Best pivot?

- Median
- Halve each time



- Worst pivot?

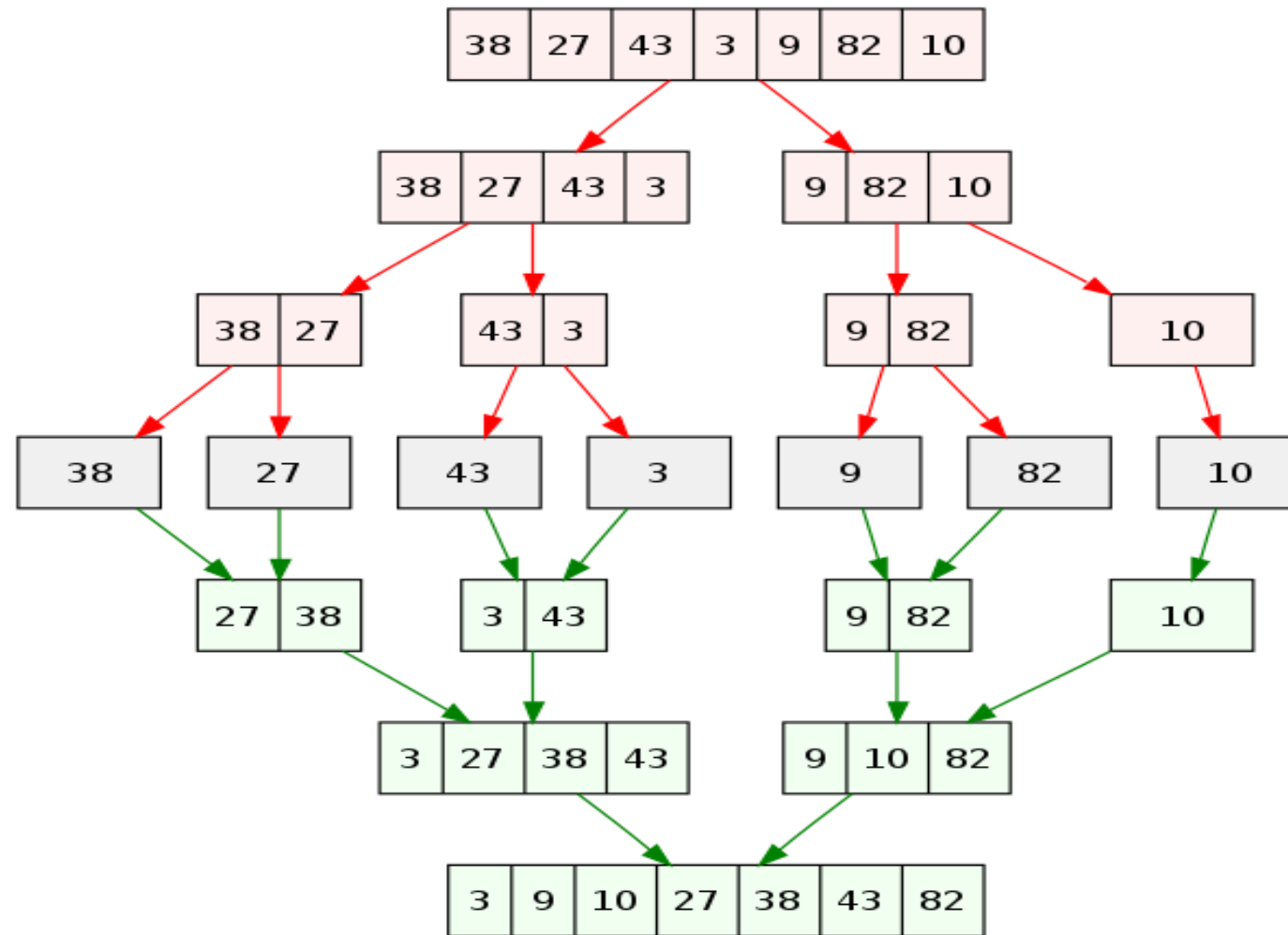
- Greatest/least element
- Problem of size $n - 1$
- $O(n^2)$



Quicksort vs. Heapsort

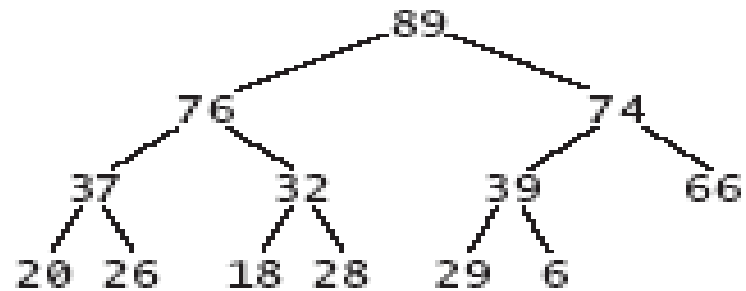
	Quicksort	Heapsort
Advantages	Fast. On average, Quicksort runs in $O(n \log n)$ time	- Fast: Heapsort always runs in $O(n \log n)$ time - Space efficient: Heapsort takes $O(1)$ additional space
Disadvantages	- Slow Worst-Case. Although it isn't common, Quicksort can take $O(n^2)$ in the worst case - Lame implementation may result in stack overflow error, since it may require $O(n)$ nested recursive calls	Slow in practice. While the asymptotic complexity of heap sort makes it look faster than quicksort, in real systems heap sort is often slower
Use cases	Studies have found quicksort to perform better on small datasets	HeapSort can be useful where memory usage is limited, such as in embedded systems

Merge Sort Example

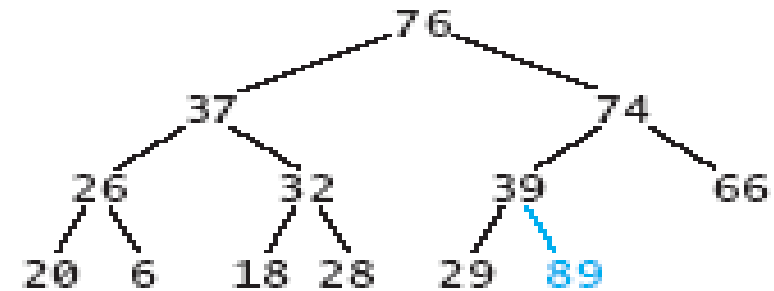


Heap Sort Example

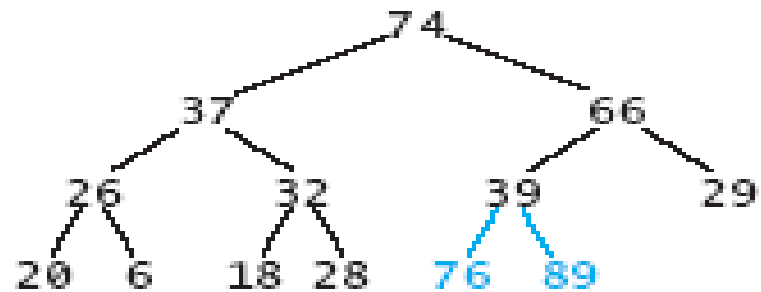
Example of a Heap with Largest Value in Root



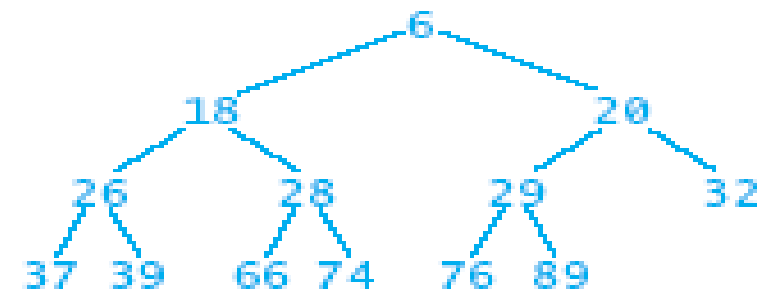
Heap After Removal of Largest Item



Heap After Removal of Two Largest Items



Heap After Removal of All Its Items



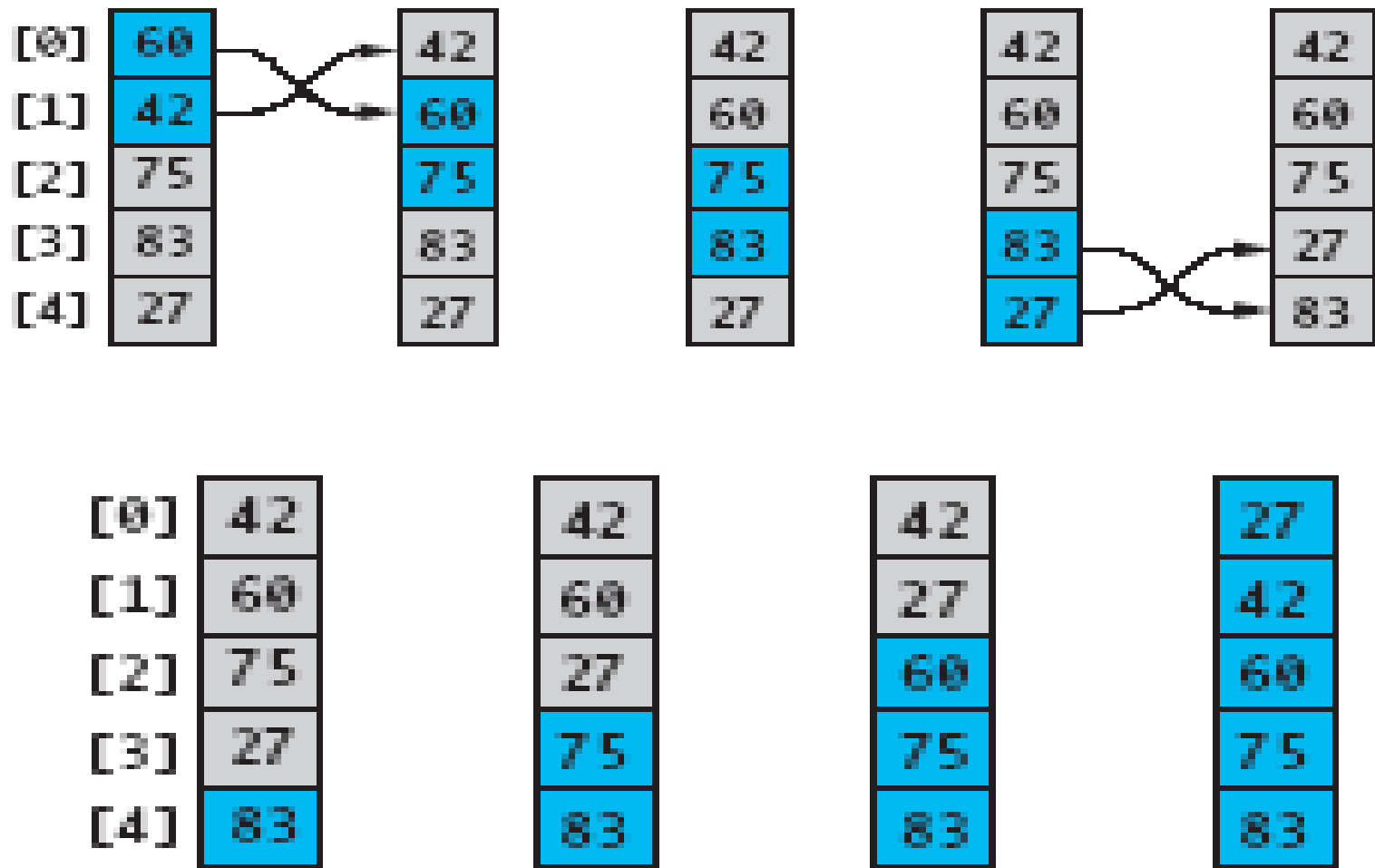
Selection Sort Example

35	65	30	60	20	scan 0-4, smallest 20 swap 35 and 20
20	65	30	60	35	scan 1-4, smallest 30 swap 65 and 30
20	30	65	60	35	scan 2-4, smallest 35 swap 65 and 35
20	30	35	60	65	scan 3-4, smallest 60 swap 60 and 60
20	30	35	60	65	done

Bubble Sort Example

- Compares adjacent array elements
 - Exchanges their values if they are out of order
- Smaller values bubble up to the top of the array
 - Larger values sink to the bottom

Bubble Sort Example



Insertion Sort Example

